
CSE D 501

*Markov Models and Reinforcement
Learning*

Taylor Kessler Faulkner

University of Washington

November 19, 2025



Admin

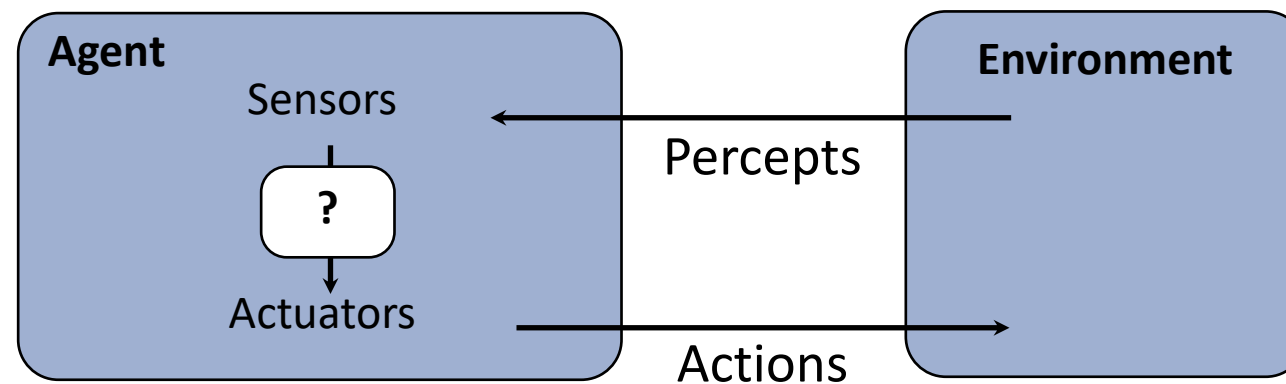
- Reminder: please tag your questions on Gradescope when submitting
 - Instructions in Canvas message
- Midterm grades & staff solutions released
- No class next week (Thanksgiving)!

Today's Class

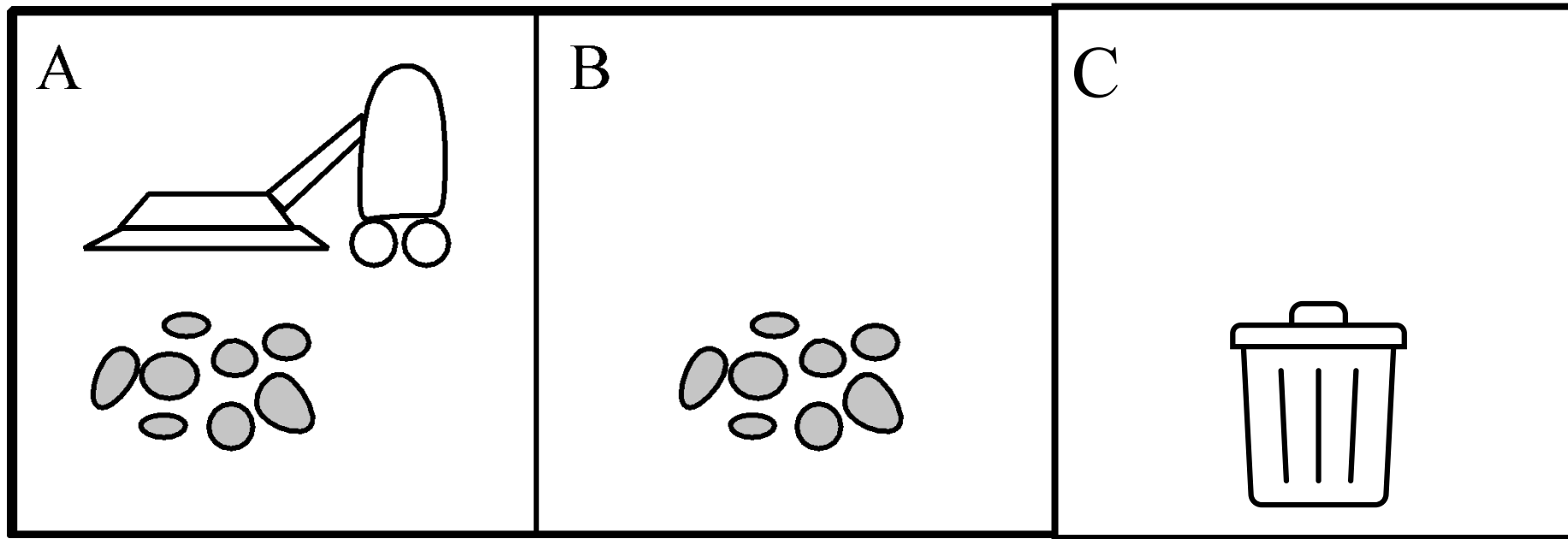
- Markov Models
- Break
- MDPs and model-based RL
- Break
- Model-free RL and extensions

Introduction to Agents

- For algorithms like RL, we often refer to a **learning agent** and **environment**
- Rather than focusing on **data** and **labels**, we think about an **agent** (a computer, a robot, a person) taking **actions** within an **environment**
- An agent **perceives** its environment through **sensors** and **acts** upon it through **actuators** (or *effectors*, depending on whom you ask)



Example: Vacuum world

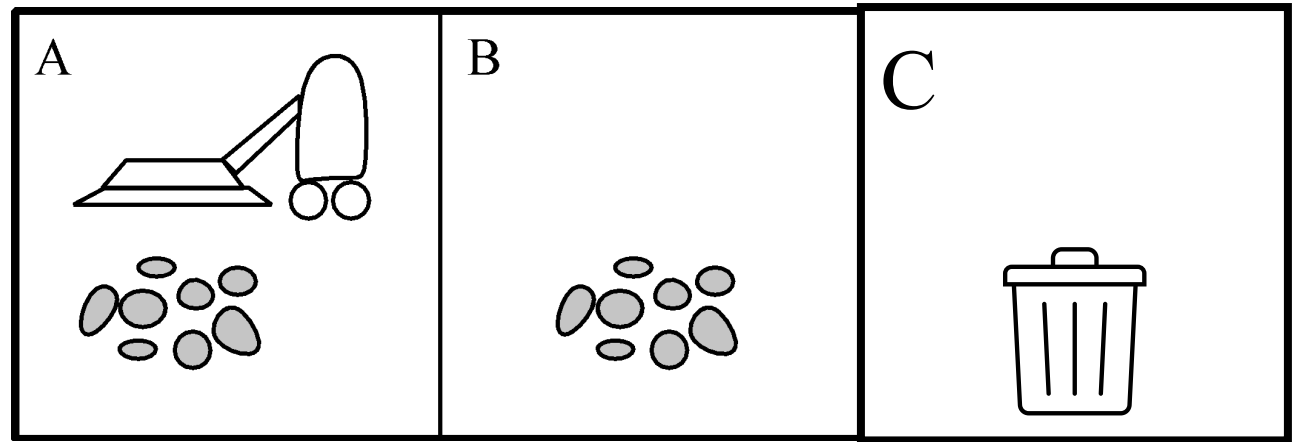


- State: [location,status], e.g., [A,*Dirty*]
- Actions: *Left*, *Right*, *Vacuum*, *Empty*, *NoOp* (do nothing)

Vacuum cleaner agent

Example agent function

State sequence	Action
[A,Clean]	Right
[A,Dirty]	Vacuum
[B,Clean]	Left
[B,Dirty]	Vacuum
[A,Clean],[B,Clean]	Left
[A,Clean],[B,Dirty]	Vacuum
...	...



What is the *right* agent function?

Rationality

- A fixed ***performance measure*** evaluates the environment sequence
- A ***rational agent*** chooses whichever action maximizes the ***expected*** value of the performance measure
 - Given the state sequence to date and prior knowledge of environment
- Will one point per square cleaned up make a rational agent behave like we want?

Rationality

- A fixed ***performance measure*** evaluates the environment sequence
- A ***rational agent*** chooses whichever action maximizes the ***expected*** value of the performance measure
 - Given the state sequence to date and prior knowledge of environment
- Will one point per square cleaned up make a rational agent behave like we want?
 - No, this rewards an agent who dumps dirt and cleans it up
 - How to improve this performance measure?

Rationality

- A fixed ***performance measure*** evaluates the environment sequence
- A ***rational agent*** chooses whichever action maximizes the ***expected*** value of the performance measure
 - Given the state sequence to date and prior knowledge of environment
- Will one point per square cleaned up make a rational agent behave like we want?
 - No, this rewards an agent who dumps dirt and cleans it up
 - How to improve this performance measure?
 - Lots of ways!
 - Only give the agent 10 points when the whole floor is clean and end the task
 - Give the agent 1 point every time a square that has never been cleaned is cleaned up
 - And more

Rationality

- Are rational agents ***omniscient***?
 - No – they are limited by the available states
- Are rational agents ***clairvoyant***?
 - No – they may lack knowledge of the environment dynamics
- Do rational agents ***explore*** and ***learn***?
 - Yes – in unknown environments these are essential
- Do rational agents ***make mistakes***?
 - No – but their actions may be unsuccessful
- Are rational agents ***autonomous*** (i.e., transcend initial program)?
 - Yes – as they learn, their behavior depends more on their own experience

Introduction to Graphical Models

One kind of Graphical Model: Markov Models

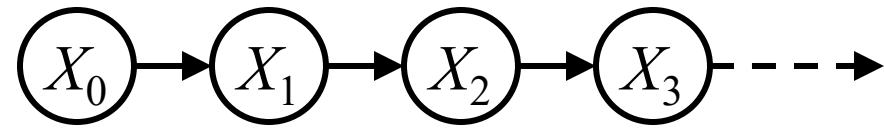
- Graphical models let us represent the joint probability distribution of a set of variables using a graph where nodes are variables and edges represent conditional dependencies
- Often, we want to reason about a **sequence** of observations
 - Speech recognition
 - Robot localization
 - Medical monitoring
- Markov models let us introduce time by modeling the edges in a graph on **transition probabilities**
 - The probability of moving from one state to another

Example: Web browsing

- State: URL visited at time t
- Transition model:
 - With probability p , choose an outgoing link at random
 - With probability $(1-p)$, choose an arbitrary new page
- Question we might ask: What is the ***stationary distribution*** over pages?
 - i.e., if an agent surfs the web forever, what fraction of time does it spend in any given page?
- Application: Google page rank

Markov Models (aka Markov chain/process)

- Value of X at a given time is called the **state** (usually discrete, finite)



$$P(X_0)$$

$$P(X_t | X_{t-1})$$

- The **transition model** $P(X_t | X_{t-1})$ specifies how the state evolves over time
- **Stationarity** assumption: transition probabilities are the same at all times
- **Markov** assumption: “future is independent of the past given the present”
 - X_{t+1} is independent of $X_0, \dots, X_{t-1}$ given X_t
 - This is a **first-order** Markov model (a k th-order model allows dependencies on k earlier steps)
- Joint distribution $P(X_0, \dots, X_T) = P(X_0) \prod_t P(X_t | X_{t-1})$

Example: Tiny Web Browsing

- States {YouTube, Reddit}

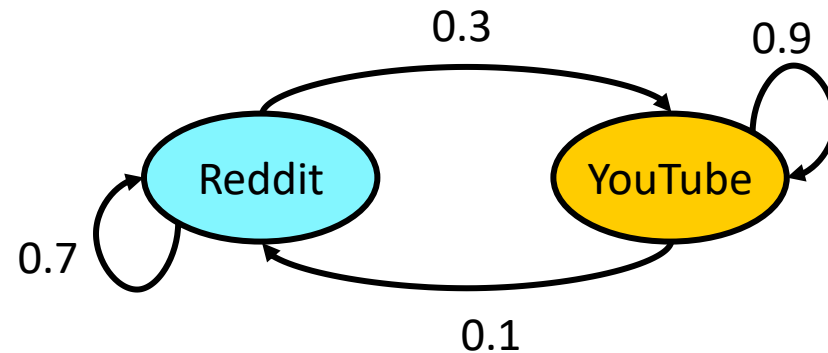
- Initial distribution $P(X_0)$

$P(X_0)$	
YouTube	Reddit
0.5	0.5

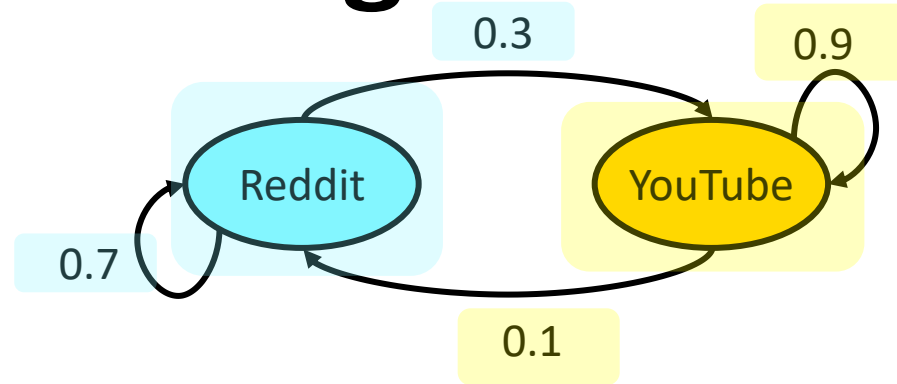
- Transition model $P(X_t | X_{t-1})$

X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

Graph Representation of problem



Tiny Web Browsing



- Time 0: $\langle 0.5, 0.5 \rangle$

$P(X_0)$	
YouTube	Reddit
0.5	0.5

- What site is our agent on at time 1?

$$\begin{aligned} \bullet \quad P(X_1) &= \sum_{x_0} P(X_1, X_0=x_0) \\ &= \sum_{x_0} P(X_0=x_0) P(X_1 | X_0=x_0) \\ &= 0.5[0.9, 0.1] + 0.5[0.3, 0.7] \\ &= [0.45, 0.5] + [0.15, 0.35] = [0.6, 0.4] \end{aligned}$$

X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

Tiny Web Browsing

- Time 1: <0.6,0.4>

- What site is our agent on at time 2?

- $$\begin{aligned} P(X_2) &= \sum_{x_1} P(X_2, X_1=x_1) \\ &= \sum_{x_1} P(X_1=x_1) P(X_2 | X_1=x_1) \\ &= 0.6<0.9,0.1> + 0.4<0.3,0.7> = <0.66,0.34> \end{aligned}$$

X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

Tiny Web Browsing

- Time 2: $\langle 0.66, 0.34 \rangle$

- What site is our agent on at time 3?

- $$\begin{aligned} P(X_3) &= \sum_{x_2} P(X_3, X_2=x_2) \\ &= \sum_{x_2} P(X_2=x_2) P(X_3 | X_2=x_2) \\ &= 0.66 \langle 0.9, 0.1 \rangle + 0.34 \langle 0.3, 0.7 \rangle = \langle 0.696, 0.304 \rangle \end{aligned}$$

X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

Forward algorithm (simple form)

- What is the state at time t ?

$$\begin{aligned} \bullet P(X_t) &= \sum_{x_{t-1}} P(X_t, X_{t-1}=x_{t-1}) \\ &= \sum_{x_{t-1}} P(X_{t-1}=x_{t-1}) P(X_t | X_{t-1}=x_{t-1}) \end{aligned}$$

Probability from
previous iteration

Transition model

- Iterate this update starting at $t=0$

And the same thing in linear algebra

- What site is our agent on at time 2?

$$\mathbf{P}(X_2) = 0.6\langle 0.9, 0.1 \rangle + 0.4\langle 0.3, 0.7 \rangle = \langle 0.66, 0.34 \rangle$$

- In matrix-vector form:

$$\mathbf{P}(X_2) = \begin{bmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{bmatrix} \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.9(0.6) + 0.3(0.4) \\ 0.1(0.6) + 0.7(0.4) \end{bmatrix} = \begin{bmatrix} 0.66 \\ 0.34 \end{bmatrix}$$

- I.e., multiply by T^T , transpose of transition matrix

X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

Stationary Distributions

- The limiting distribution is called the **stationary distribution** P_∞ of the chain
- It satisfies $P_\infty = P_{\infty+1} = T^T P_\infty$
- Solving for P_∞ in the example:

$$\begin{bmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{bmatrix} \begin{bmatrix} p \\ 1-p \end{bmatrix} = \begin{bmatrix} p \\ 1-p \end{bmatrix}$$

$$0.9p + 0.3(1-p) = p$$

$$p = 0.75$$

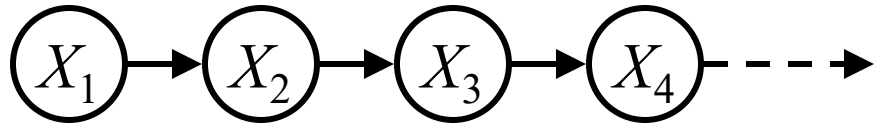
$$1-p = 0.25$$

Stationary distribution is $\langle 0.75, 0.25 \rangle$ **regardless of starting distribution**

In our example, that means our agent will spend 75% of its time on YouTube, and 25% on Reddit

Stationary Distributions

- Question: What's $\mathbf{P}(X)$ at time $t = \text{infinity}$?



X_{t-1}	$P(X_t X_{t-1})$	
	YouTube	Reddit
YouTube	0.9	0.1
Reddit	0.3	0.7

$$P_{\infty}(\text{YouTube}) = P(\text{YouTube}|\text{YouTube})P_{\infty}(\text{YouTube}) + P(\text{YouTube}|\text{Reddit})P_{\infty}(\text{Reddit})$$

$$P_{\infty}(\text{Reddit}) = P(\text{Reddit}|\text{YouTube})P_{\infty}(\text{YouTube}) + P(\text{Reddit}|\text{Reddit})P_{\infty}(\text{Reddit})$$

$$P_{\infty}(\text{YouTube}) = 0.9P_{\infty}(\text{YouTube}) + 0.3P_{\infty}(\text{Reddit})$$

$$P_{\infty}(\text{Reddit}) = 0.1P_{\infty}(\text{YouTube}) + 0.7P_{\infty}(\text{Reddit})$$

$$P_{\infty}(\text{YouTube}) = 3P_{\infty}(\text{Reddit})$$

$$P_{\infty}(\text{Reddit}) = \frac{1}{3}P_{\infty}(\text{YouTube})$$

Also: $P_{\infty}(\text{YouTube}) + P_{\infty}(\text{Reddit}) = 1$



$$P_{\infty}(\text{YouTube}) = 0.75, P_{\infty}(\text{Reddit}) = 0.25$$

Break

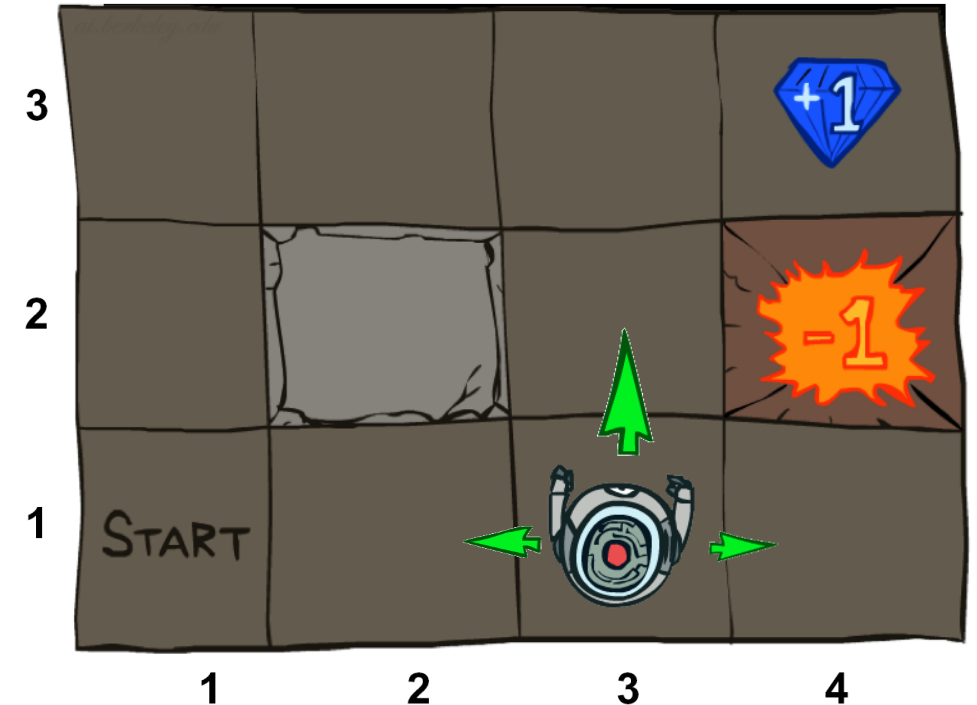
Get some water, stretch your legs, chat, be back in 10

Markov Decision Processes

- Markov Decision Processes (MDPs) extend Markov Models by adding **actions** and **rewards**
- This lets the **agent** affect the environment and receive information about those actions
- We'll study MDPs in terms of a **gridworld** problem
 - Generally not very realistic, divides the world into “grids”
 - No continuous values: for example, location is represented by (x_1, x_2) where $x_1 \in [0,1,2,3]$ and $x_2 \in [0,1,2,3,4,5]$
 - Deep RL can handle continuous states/actions

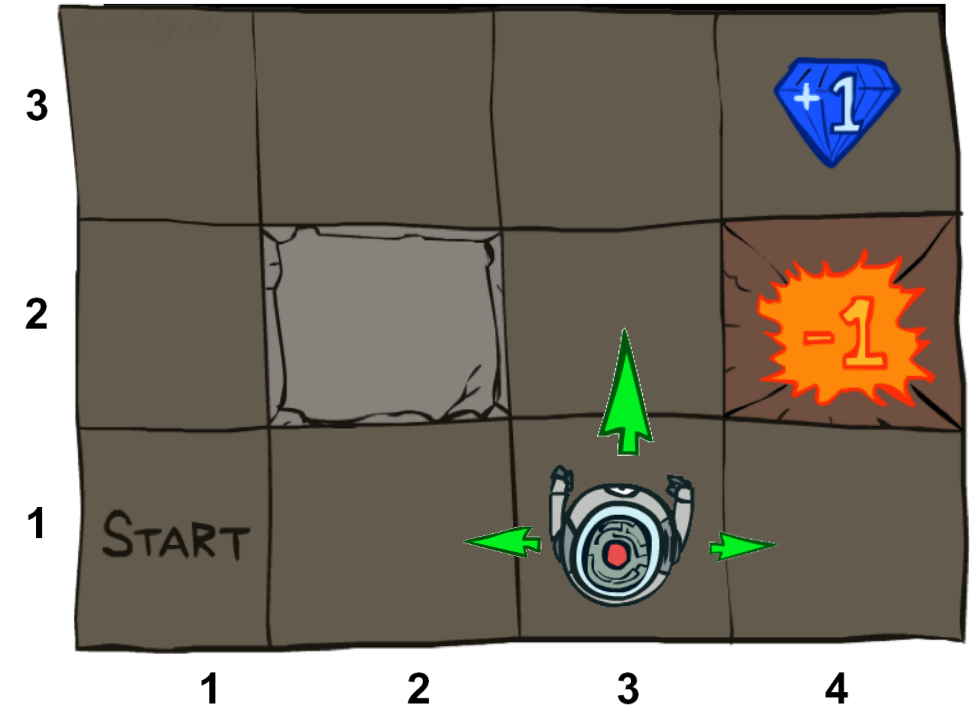
Example: Grid World

- A maze-like problem
 - The agent lives in a grid
 - Walls block the agent's path
 - The agent can go in cardinal directions (up, down, left, right) – no diagonal movement
- The agent receives rewards each time step
 - Small “living” reward r each step (often negative)
 - Big rewards tend to come at the end (good or bad)
- Goal: maximize sum of rewards



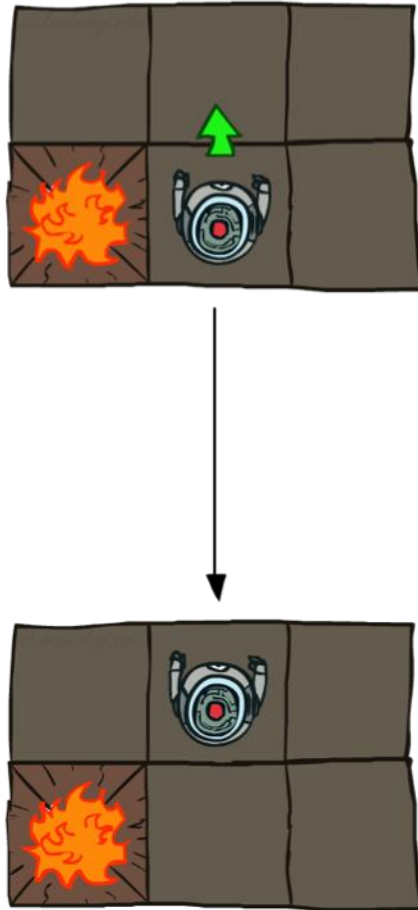
Example: Grid World

- Let's say:
 - Living reward = -0.1
 - Reward for reaching gem: 1
 - Reward for stepping on fire: -1
- Say the agent chooses to go **up, up, right**
 - What is the sum of rewards that the robot receives from its current position?
- Say the agent chooses to go **up, up, left, right, right**
 - What is the sum of rewards that the robot receives from its current position?
- Why do we use a negative living reward here?

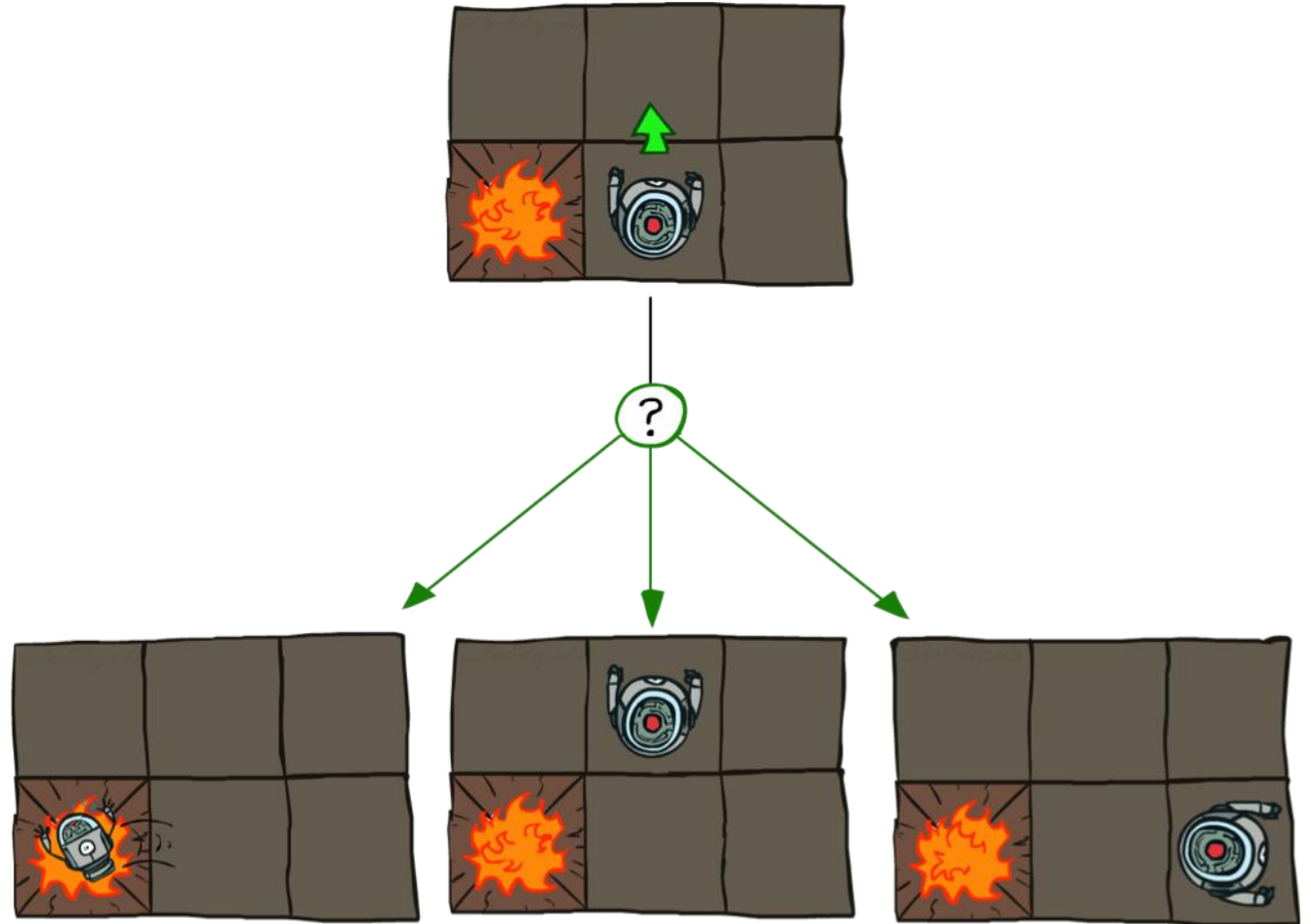


Grid World Actions

Deterministic Grid World

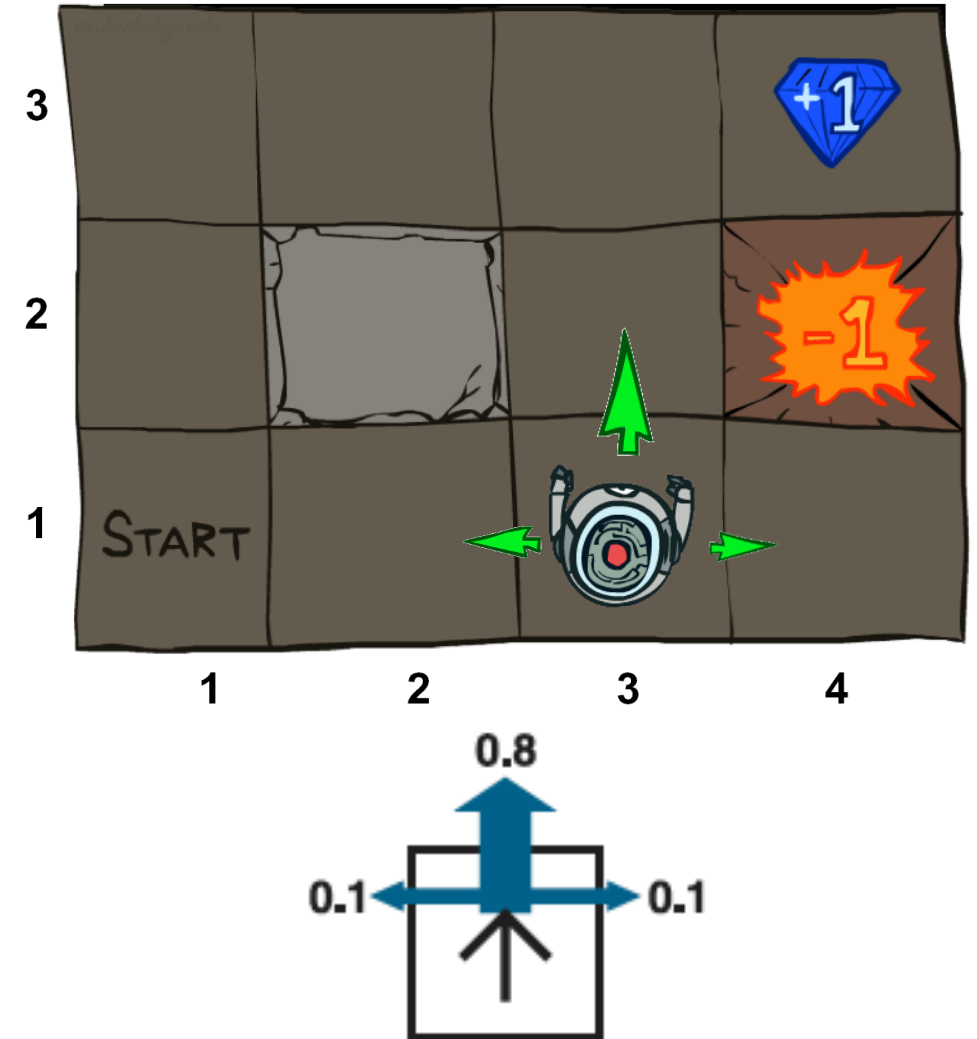


Stochastic Grid World



Example: Grid World

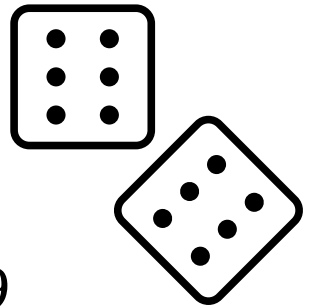
- Noisy movement: actions do not always go as planned
 - 80% of the time, the agent takes the correct action
 - 10% of the time, the agent ends up going 90 degrees left; 10% 90 degrees right
 - If there is a wall in the direction the agent takes, the agent stays put
- Can we still calculate the sum of rewards the agent will receive after taking actions?
 - We can calculate the **expected** sum of rewards!



Example: Expectation

- The **expected value** of a variable is a weighted sum over all possible values
- It is **not** the value that is most often taken
- Example: weighted die roll
 - You have a weighted die that is 30% likely to land on 6
 - The other numbers have even odds: $70\%/5 = 14\%$ likely to land on each other number
 - Let X be the variable denoting the result of a random die roll
 - While this value is most likely to be 6, the **expected value** is

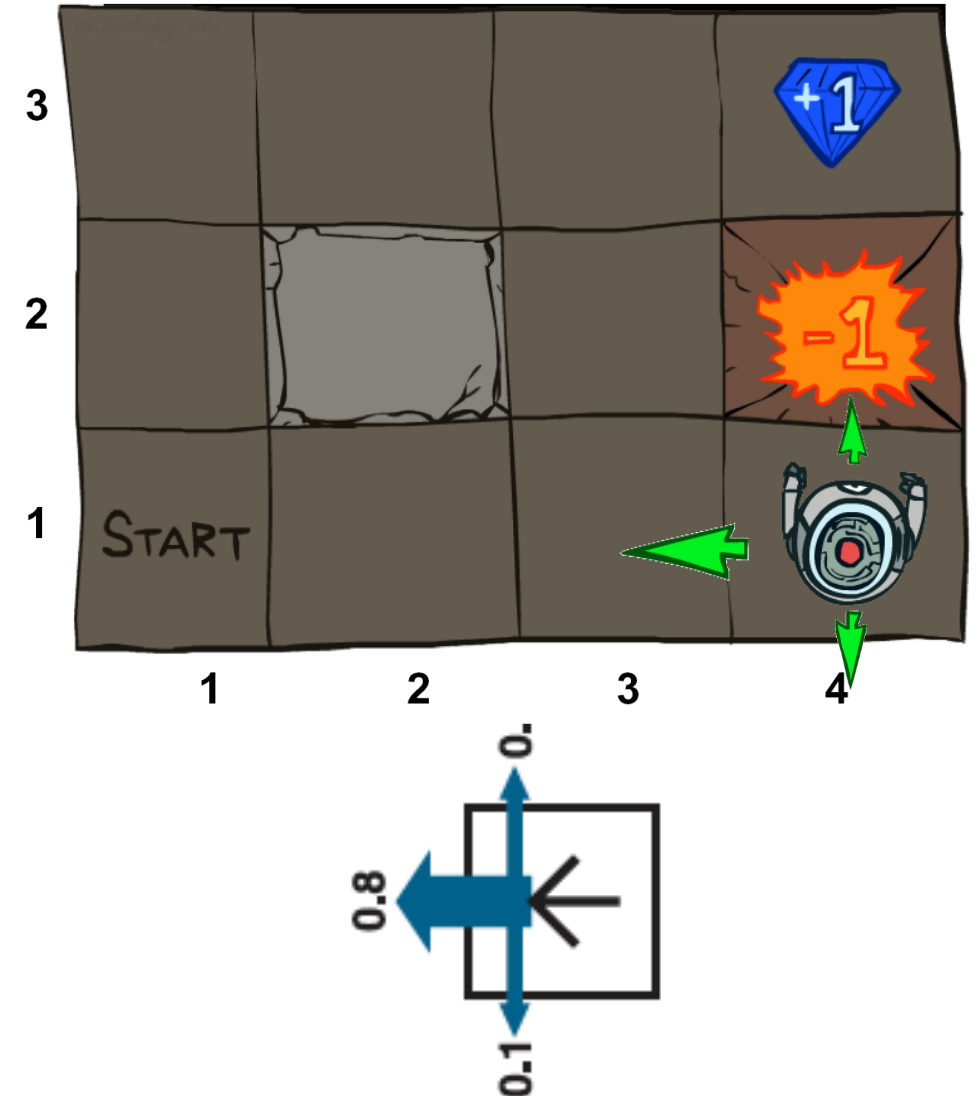
$$\mathbb{E}[X] = 0.3(6) + 0.14(5) + 0.14(4) + 0.14(3) + 0.14(2) + 0.14(1) = 3.9$$



- Notice, this is not actually a number on the die

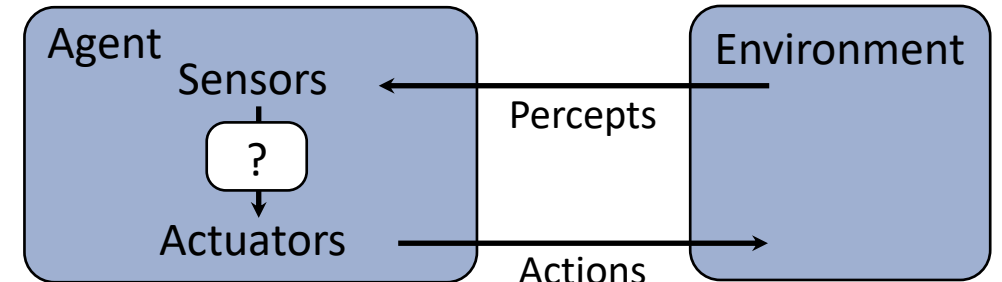
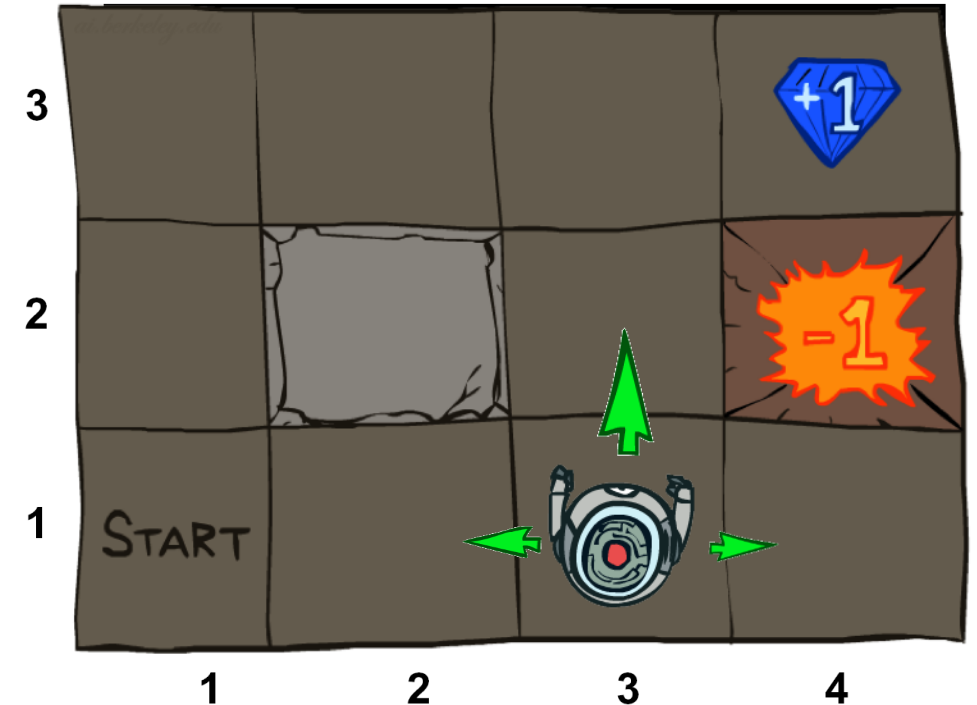
Example: Grid World

- What is the expected next reward if the agent attempts to take steps **left** from this square?
 - Living reward = -0.1
 - Reward for reaching gem: 1
 - Reward for stepping on fire: -1
- $0.8(-0.1) + 0.1(-0.1) + 0.1(-1) = -0.19$



Markov Decision Process (MDP)

- An MDP is defined by:
 - A set of **states** $s \in S$
 - A set of **actions** $a \in A$
 - A **transition model** $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s' | s, a)$
 - A **reward function** $R(s, a, s')$ for each transition
 - Can also be defined as $R(s, a)$ or $R(s')$
 - A **start state**
 - Possibly a **terminal state** (or **absorbing state**)
 - **Value function** which is additive (discounted) rewards



Markov Decision Process (MDP)

- Environment history: $[s_0, a_0, s_1, a_1, \dots, s_t]$
- “Markov assumption” or “Markov property” generally means that given the present state, the future and the past are independent
- For Markov decision processes, “Markov” means action outcomes depend only on the current state

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0)$$

=

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$



Andrey Markov
(1856-1922)

Markov Decision Process (MDP)

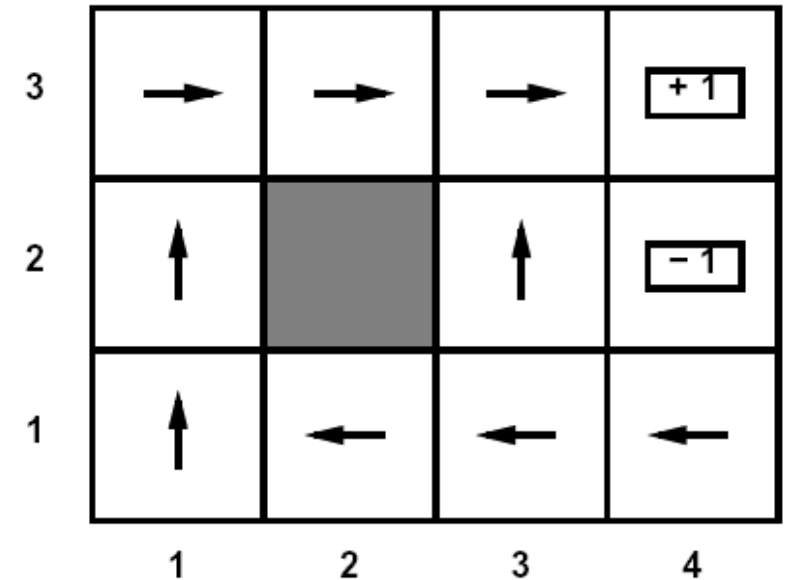
- What if we added something to our environment, like a sprinkler that turns on after 10 time steps and gets the containing square wet?
- We might want to robot to avoid the square with the sprinkler once it is on, but not before
- If our state definition is just the position of the robot (x_1, x_2) , can the robot make the best decision only considering the last state?
- How could we extend our state to allow the robot to only need to consider the last state?



Andrey Markov
(1856-1922)

Policies

- A policy π gives an action for each state, $\pi: S \rightarrow A$
- For MDPs, we want an optimal **policy** $\pi^*: S \rightarrow A$
 - An optimal policy maximizes value
 - **Value:** expected sum of rewards from current state



Sample Optimal Policies

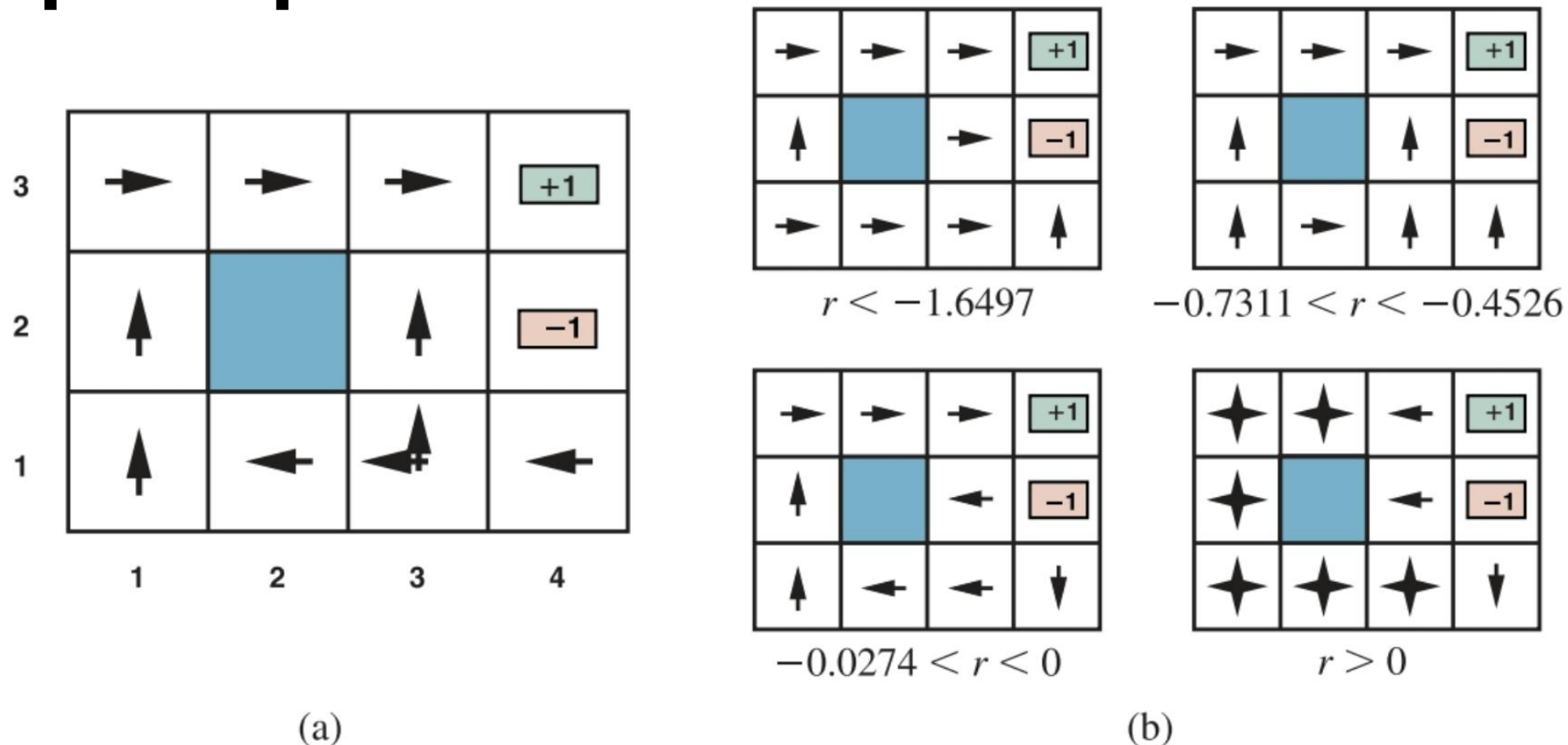


Figure 17.2 (a) The optimal policies for the stochastic environment with $r = -0.04$ for transitions between nonterminal states. There are two policies because in state (3,1) both *Left* and *Up* are optimal. (b) Optimal policies for four different ranges of r .

Utilities of Sequences

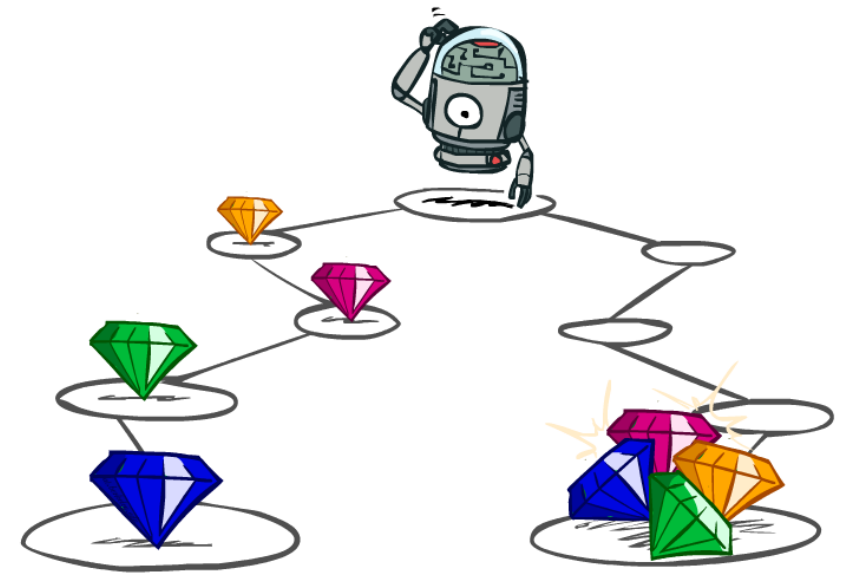
- What preferences should an agent have over reward sequences?

- More or less?

[1, 2, 2] or [2, 3, 4]

- Now or later?

[0, 0, 1] or [1, 0, 0]



Discounting

- Instead of treating all rewards the same, we will **discount** rewards related to the time they will be received

$$V_h([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$$

- ***Additive discounted utility***: where $\gamma \in [0, 1]$ is the ***discount factor***

Practice: Discounting

- Given:

10				1
----	--	--	--	---

a	b	c	d	e
---	---	---	---	---

 - Actions: East, West, and Exit where Exit is only available in exit states a, e
 - Transitions: deterministic
- 1: For $\gamma = 1$, what is the optimal policy?

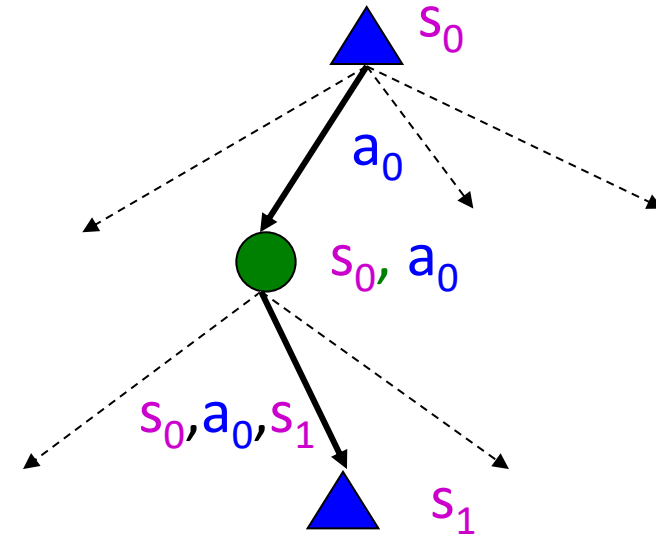
10				1
----	--	--	--	---
- 2: For $\gamma = 0.1$, what is the optimal policy?

10				1
----	--	--	--	---
- 3: For which γ are West and East equally good when in state d?

The value of a policy

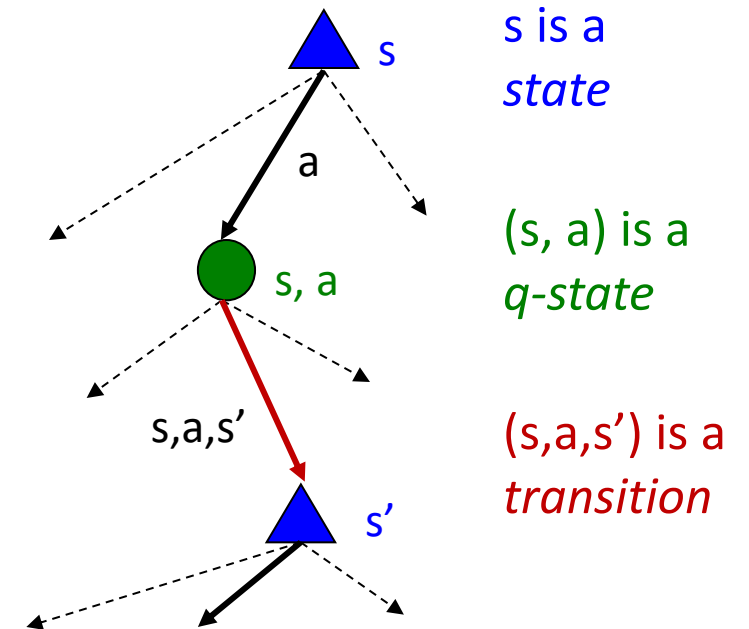
- Executing a policy π from any state s_0 generates a sequence
 $s_0, \pi(s_0), s_1, \pi(s_1), s_2, \dots$
- This corresponds to a sequence of rewards
 $R(s_0, \pi(s_0), s_1), R(s_1, \pi(s_1), s_2), \dots$
- This reward sequence happens with probability
 $P(s_1 | s_0, \pi(s_0)) \times P(s_2 | s_1, \pi(s_1)) \times \dots$
- The value of π in s_0 is written $V^\pi(s_0)$
 - It's the sum over all possible state sequences of
(discounted sum of rewards) $\times$ (probability of state sequence)

$$V^\pi(s_0) = E \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t), s_{t+1}) \right]$$



Optimal Quantities

- The optimal policy:
 $\pi^*(s)$ = optimal action from state s
Gives highest $V^\pi(s)$ for any π
- The value of a state s :
 $V^*(s) = V^{\pi^*}(s)$ = expected utility starting in s
and acting optimally
- The value of a q-state (s,a) :
 $Q^*(s,a)$ = expected utility of taking action a
in state s and (thereafter) acting optimally
 $V^*(s) = \max_a Q^*(s,a)$



Bellman equations (Shapley, 1953)

- The value/utility of a state is
 - The expected reward for the next transition plus the discounted value/utility of the next state, assuming the agent chooses the optimal action
- Hence we have a recursive definition of value (Bellman equation):

$$V(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

- Similarly, Bellman equation for Q-functions

$$\begin{aligned} Q(s, a) &= \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \\ &= \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \end{aligned}$$

Bellman Equation

The Bellman equation states that the **value of a state is equal to the expected immediate reward plus the discounted value of the next state**

Recursively
define the **value**
of a state s

$$V(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

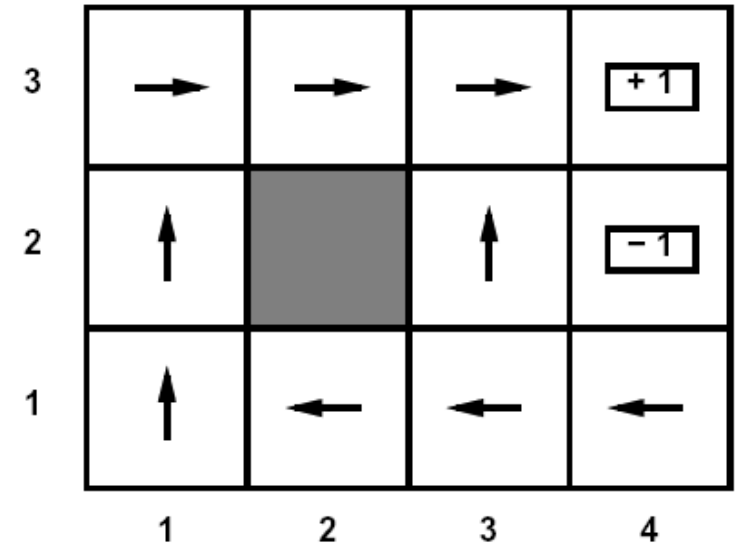
Transition
probability from s to
 s' given action a

Reward for reaching state s'
from state s with action a

Discount factor:
how much
priority is placed
on immediate
reward vs future
rewards

Value Iteration

- Now, how do we use this definition of value to determine a policy?
- We'll have to compute the value for each state
- If we know everything about the MDP, we can use **value iteration**

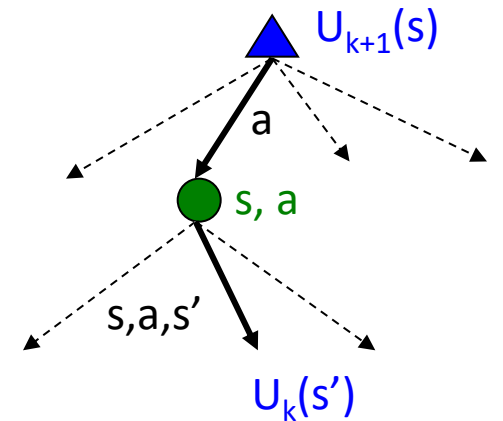


Value Iteration

- Start with (say) $V_0(s) = 0$ and some termination parameter ϵ
- Repeat until convergence (i.e., until all updates smaller than ϵ)
 - Do a **Bellman update** from each state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do



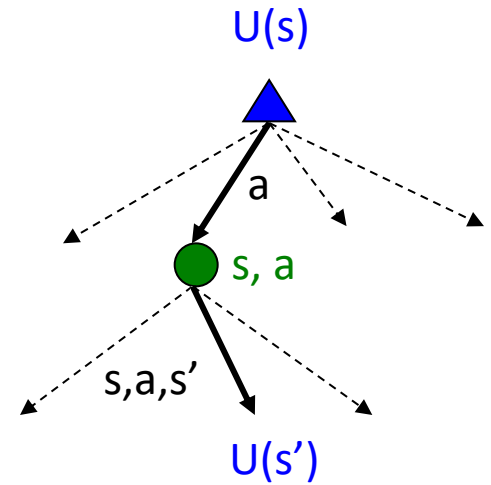
Value Iteration

- Bellman equations **characterize** the optimal values:

- $V^*(s) = \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V^*(s')]]$

- Value iteration **computes** them:

- $V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$



Value Iteration

Initialize $V(s)$ arbitrarily for all $s \in S$ (e.g., to 0)

Initialize θ to a small positive value

While $\Delta \geq \theta$:

$\Delta = 0$

for $s \in S$:

$v = V(s)$

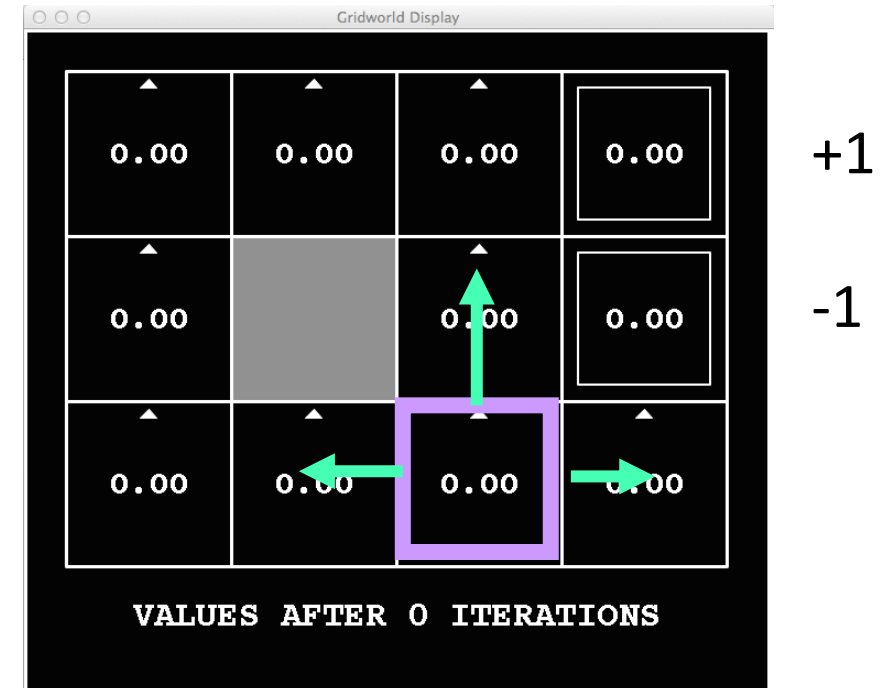
$V(s) = \max_a (\sum_{s',r} P(s'|s,a)[r + \gamma V(s')])$

$\Delta = \max(\Delta, |v - V(s)|)$

k=0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

$$V_{k+1}(s) \leftarrow 0.8[0 + 0.9(0)] + 0.1[0 + 0.9(0)] + 0.1[0 + 0.9(0)] = 0$$

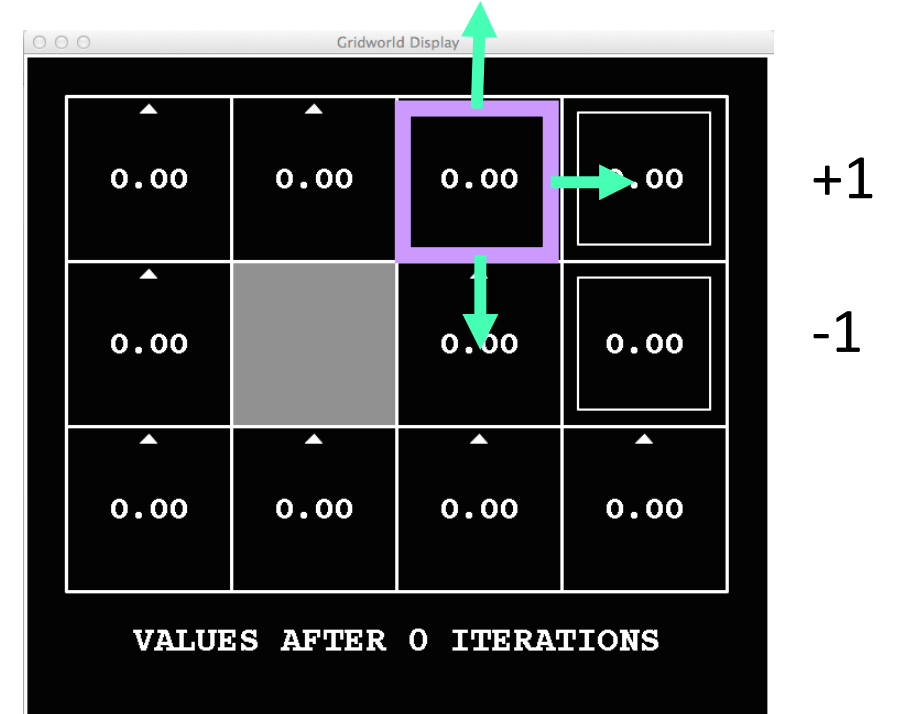


Noise = 0.2
Discount = 0.9
Living reward = 0

k=0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

$$V_{k+1}(s) \leftarrow 0.8[0 + 0.9(0)] + 0.1[0 + 0.9(0)] + 0.1[0 + 0.9(0)] = 0$$



Noise = 0.2
Discount = 0.9
Living reward = 0

k=0

Terminal state!

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

$$V_{k+1}(s) \leftarrow 1 + 0.9(0) = 1$$

What other state will get an updated value on this iteration?



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=1



Noise = 0.2
Discount = 0.9
Living reward = 0

k=0



$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

$$V_{k+1}(s) \leftarrow 0.8(0 + 0.9(1)) + 0.1(0 + 0.9(0)) + 0.1(0 + 0.9(0)) = 0.72$$

Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=2



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=3



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=4



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=5



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=6



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=7



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=8



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=9



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=10



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

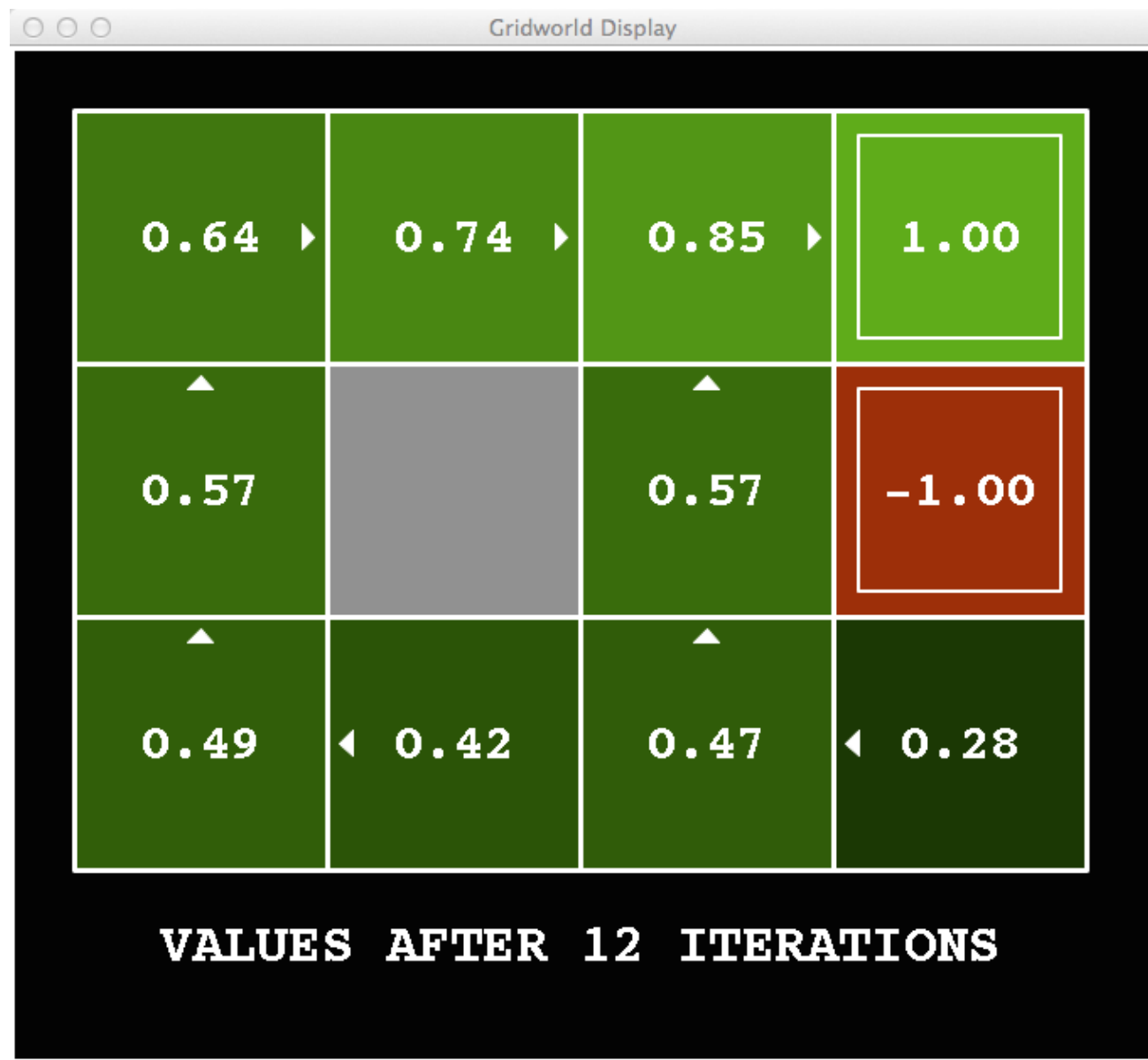
k=11



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=12



Noise = 0.2
Discount = 0.9
Living reward = 0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | a, s) [R(s, a, s') + \gamma V_k(s')]]$$

k=100



Noise = 0.2
Discount = 0.9
Living reward = 0

Policy Extraction

- How should the agent act given $V(s)$?
- Maximize expected utility! (as if V is correct)
 - That is, the next action should take you to the state with highest value
- This is called ***policy extraction***, since it finds the policy π_U implied by the values V



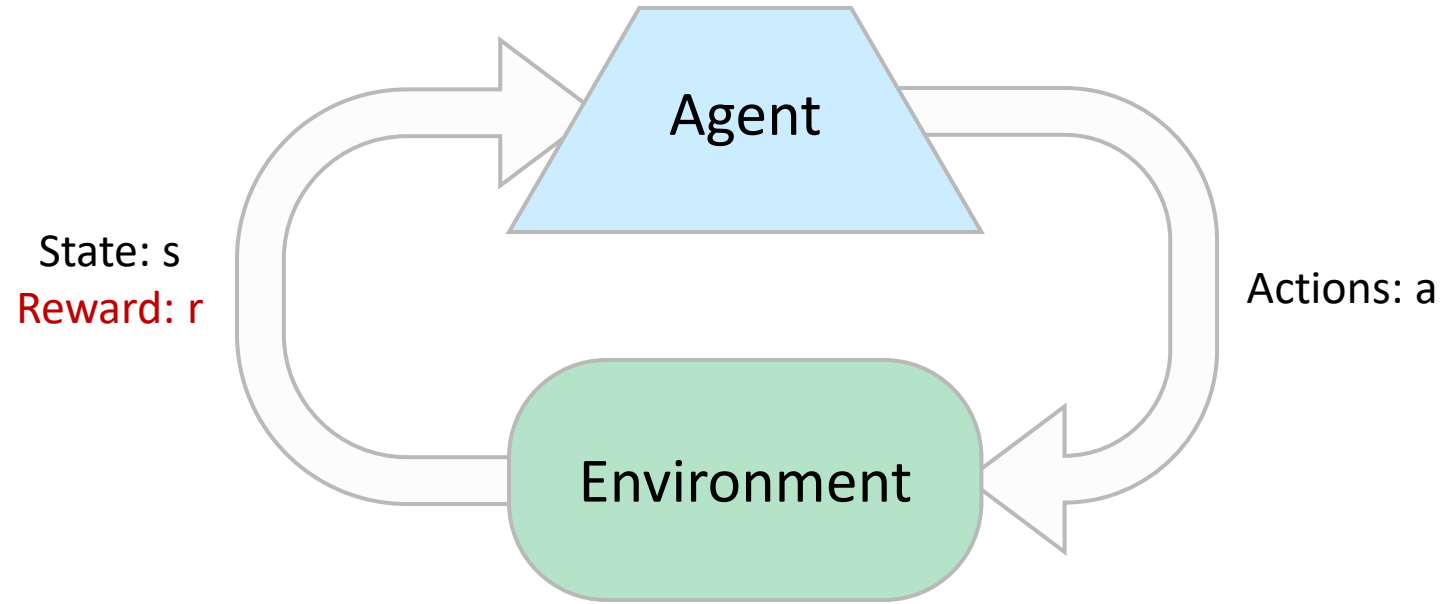
Break

Get some water, stretch your legs, chat, be back in 10

Model-Free Reinforcement Learning

- Still assume a Markov decision process (MDP):
 - A set of states $s \in S$
 - A set of actions (per state) A
 - A model $T(s,a,s')$
 - A reward function $R(s,a,s')$
- Still looking for a policy $\pi(s)$
- New twist: don't know T or R
 - i.e. we don't know which states are good or what the actions do
 - Must actually try actions and states out to learn

Reinforcement Learning



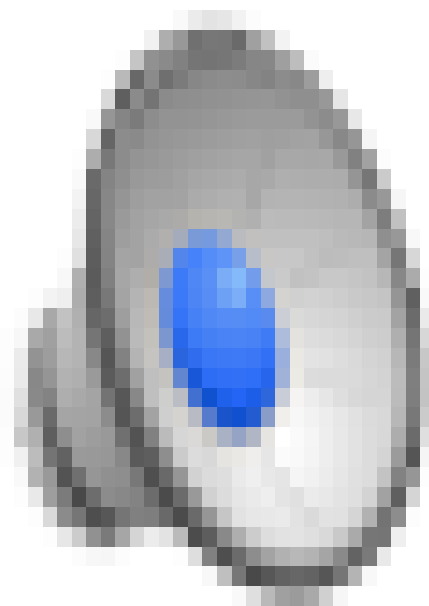
- Basic idea:
 - Receive feedback in the form of **rewards**
 - Agent's utility is defined by the reward function
 - Must (learn to) act so as to **maximize expected rewards**
 - All learning is based on observed samples of outcomes!

Reinforcement learning

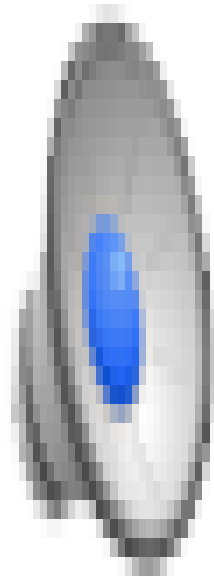
Basic ideas:

- **Exploration**: you have to *try unknown actions* to get information
- **Exploitation**: eventually, you have to use what you know
- **Sampling**: you may need to repeat many times to get good estimates
- **Generalization**: what you learn in one state may apply to others too

Video of Demo Crawler Bot



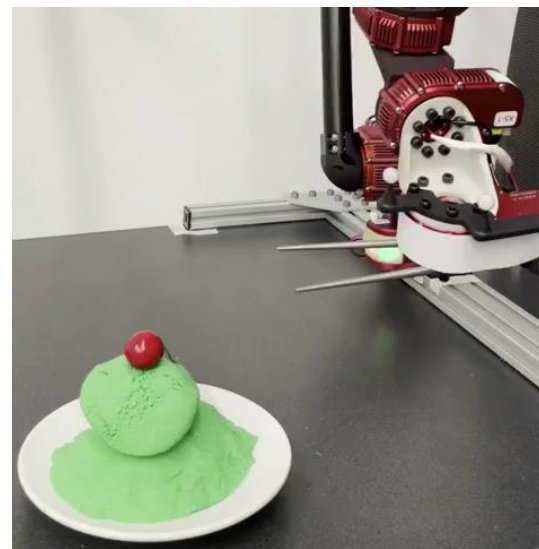
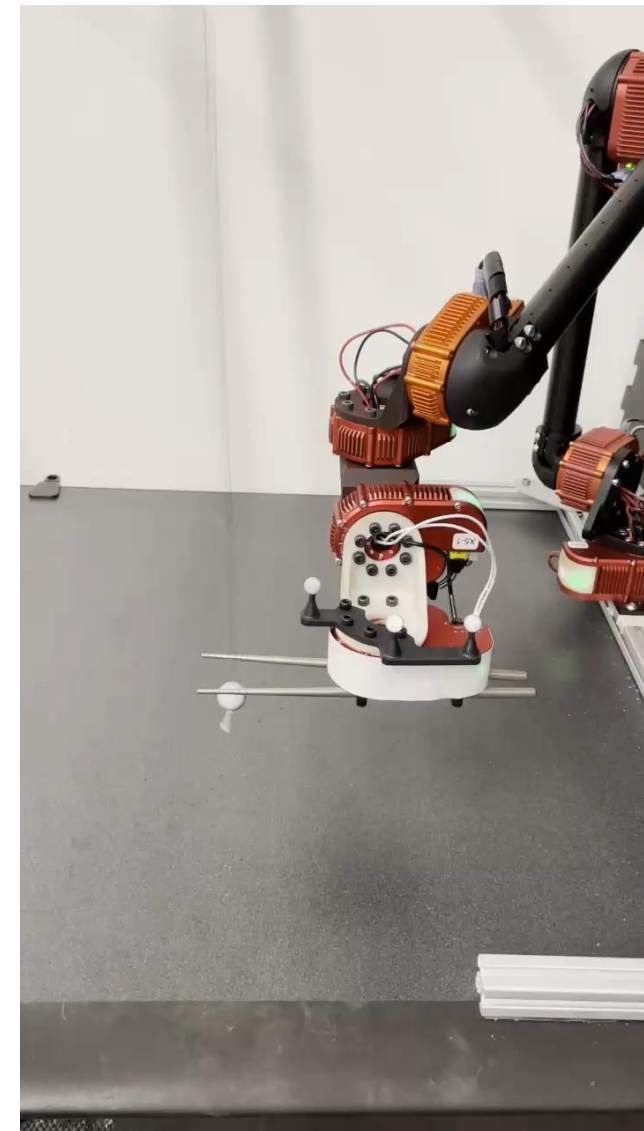
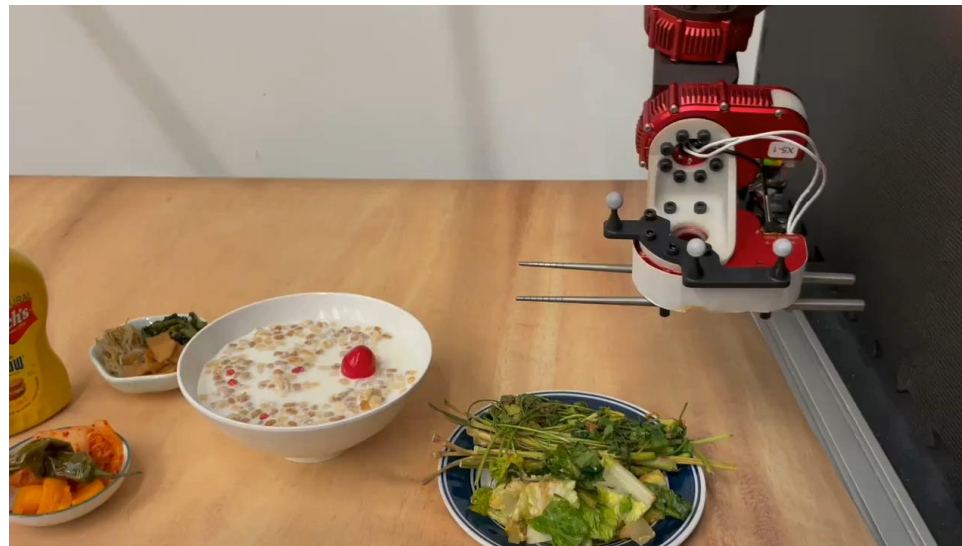
DeepMind Atari (©Two Minute Lectures)



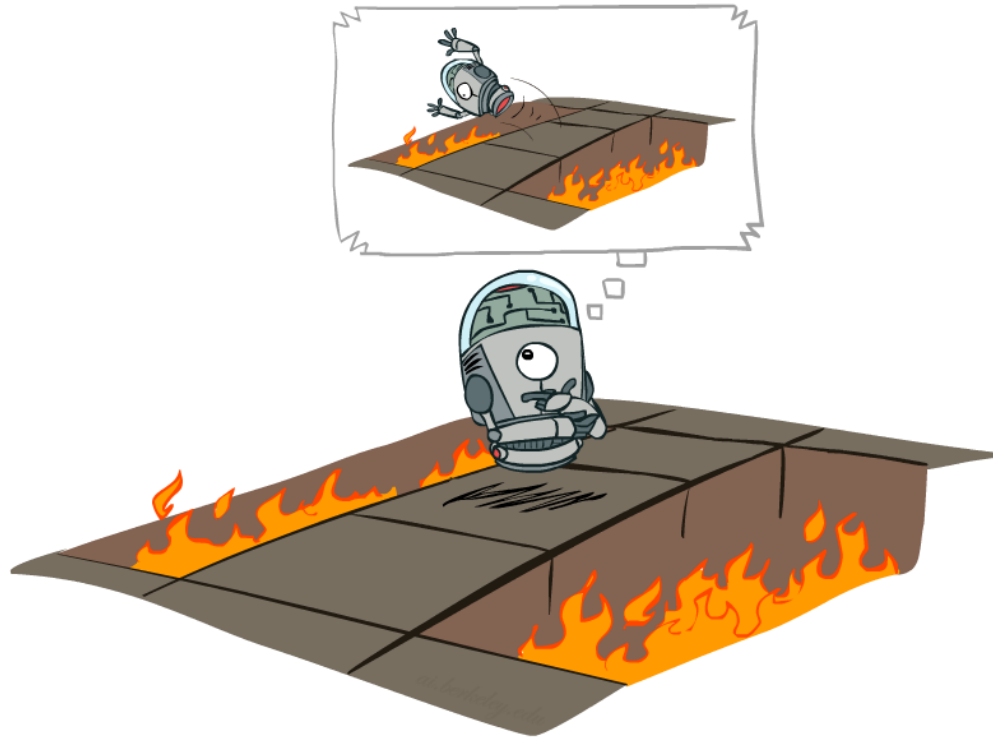
Cherry picking with Reinforcement learning

[Yunchu Zhang](#)^{1*}, [Liyiming Ke](#)^{2*}, [Abhay Deshpande](#)², [Abhishek Gupta](#)², [Siddhartha Srinivasa](#)²,

¹Carnegie Mellon University, ²University of Washington



Offline (MDPs) vs. Online (RL)



Offline Solution



Online Learning

Detour: Q-Value Iteration

- Value iteration: find successive (depth-limited) values of **states**
 - Start with $V_0(s) = 0$
 - Given V_k , calculate the depth $k+1$ values for all states:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma U_k(s')]]$$

- Q-value Iteration: find successive (depth-limited) values of **state-action pairs**
 - Start with $Q_0(s,a) = 0$
 - Given Q_k , calculate the depth $k+1$ q-values for all q-states:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma \max_{a'} Q_k(s',a')]]$$

Q-Learning

- Q-Learning: sample-based Q-value iteration

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s') [R(s,a,s') + \gamma \max_{a'} Q_k(s',a')]]$$

- Learn $Q(s,a)$ values as you go

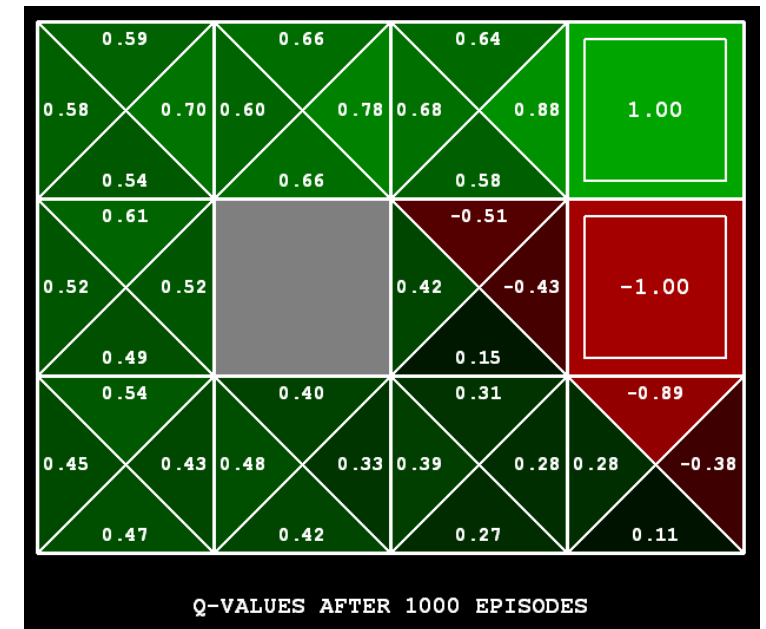
- Receive a sample (s,a,s',r)
- Consider your old estimate: $Q(s,a)$
- Consider your new sample estimate:

$$sample = R(s,\pi(s),s') + \gamma \max_{a'} Q(s',a')$$

- Incorporate the new estimate into a running average:

$$Q(s,a) \leftarrow (1 - \alpha)Q(s,a) + (\alpha) [sample]$$

Alpha is the learning rate, gamma is the discount factor



Q-Learning

Learning rate $\alpha \in (0,1]$, small positive ϵ

Initialize $Q(s, a)$ for all s, a (often small random values), except that $Q(\text{terminal states}, \cdot) = 0$

For each episode:

$s = \text{start state}$

while s is not terminal:

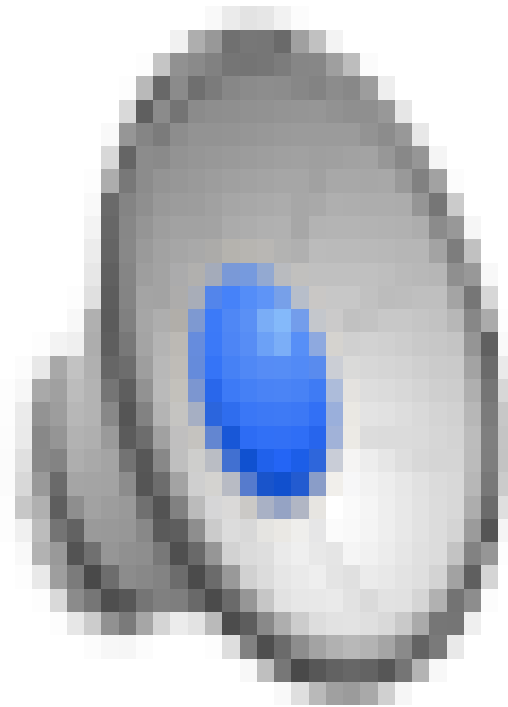
$a = \max_a(Q(s, a))$ with probability $1 - \epsilon$, random otherwise

take action a , observe s', r

$Q(s, a) = Q(s, a) + \alpha[r + \gamma \max_u Q(s', u) - Q(s, a)]$

$s = s'$

Video of Demo Q-Learning -- Gridworld

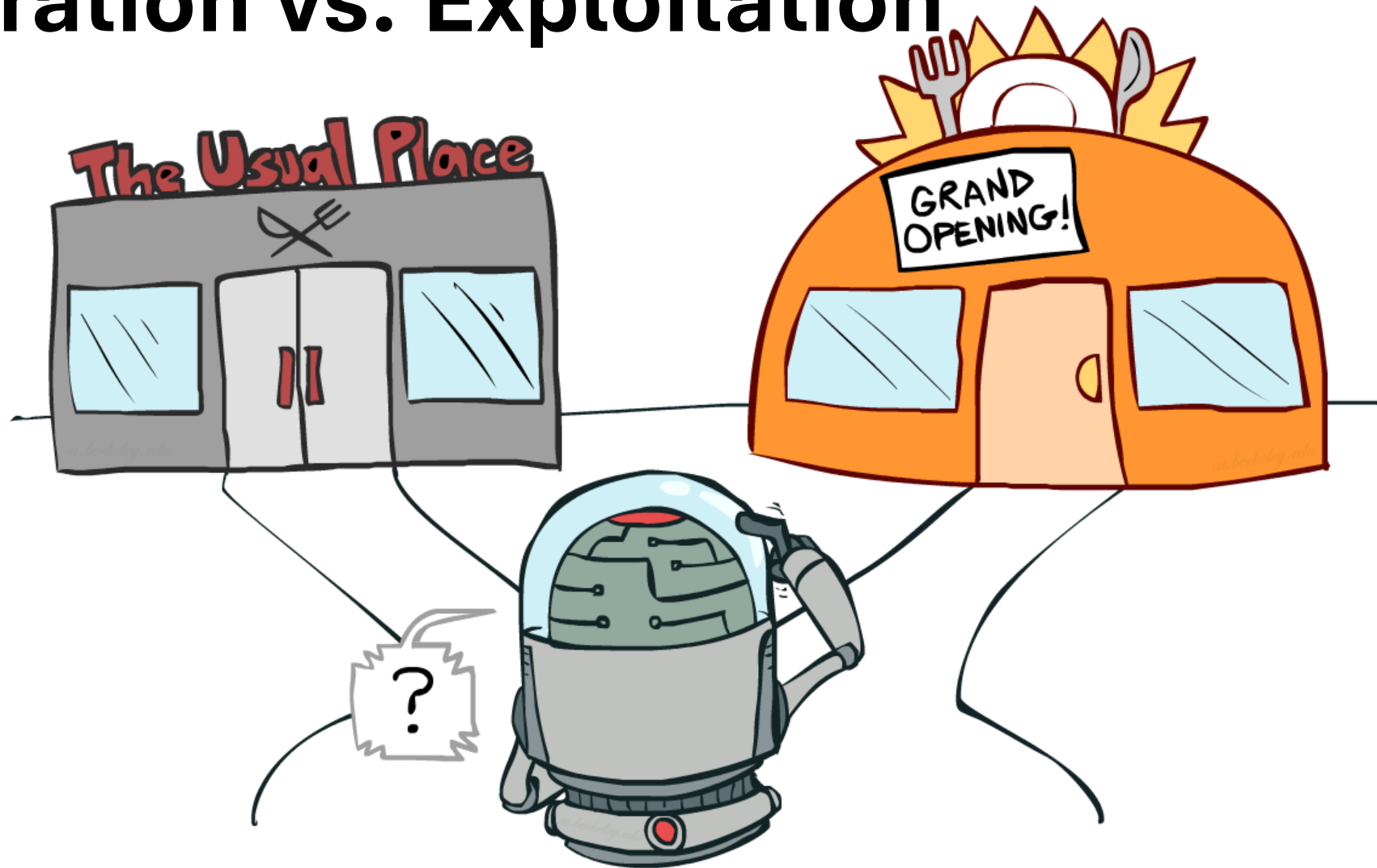


alpha = .5

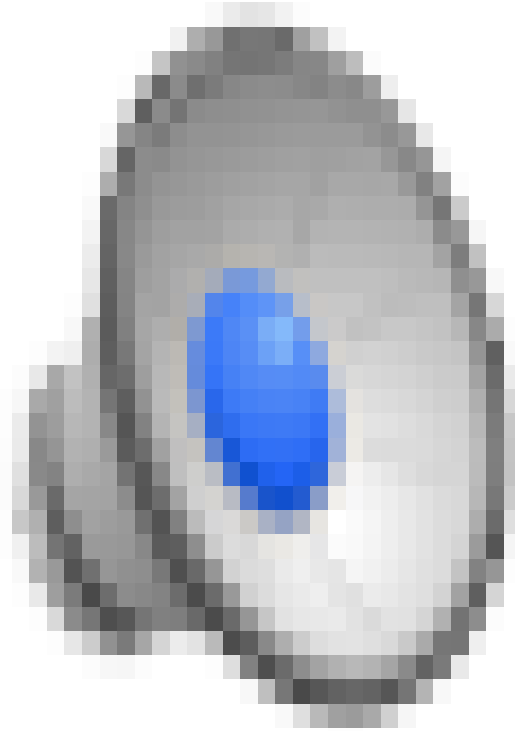
Q-Learning Properties

- Q-learning converges to optimal policy -- even if the agent is acting suboptimally!
- Caveats:
 - You have to explore enough
 - You have to eventually make the learning rate small enough
 - ... but not decrease it too quickly
 - Basically, in the limit, it doesn't matter how you select actions (!)

Exploration vs. Exploitation



Video of Demo Q-learning – Manual Exploration – Bridge Grid



How to Explore?

- Several schemes for forcing exploration
 - Simplest: random actions (ϵ -greedy)
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1-\epsilon$, act on current policy
- Problems with random actions?
 - You do eventually explore the space, but keep thrashing around once learning is done
 - One solution: lower ϵ over time
 - Another solution: exploration functions

Video of Demo Q-learning – Epsilon-Greedy – Crawler



Exploration Function Example

- When to explore?
 - Random actions: explore a fixed amount
 - Better idea: explore based on actions' Q-values
- Softmax
 - Yes, we're still using Softmax
 - Takes a **temperature** value T (controls the degree of exploration)
 - High T: more exploration
 - Low T: more exploitation
- Action selection with softmax
 - $$P(a) = \frac{e^{Q(s,a)/T}}{\sum_{a'} e^{Q(s,a')/T}}$$
 - Take any action a with probability P(a)